Zellic

Prepared for
Kelly Zhang
LayerZero Labs

Prepared by
Katerina Belotskaia
Ethan Lee
Zellic

March 27, 2026

LayerZero Console
Smart Contract Security Assessment

Contents

About Zellic **4**

1. Overview **4**

1.1. Executive Summary 5

1.2. Goals of the Assessment 5

1.3. Non-goals and Limitations 5

1.4. Results 5

2. Introduction **6**

2.1. About LayerZero Console 7

2.2. Methodology 7

2.3. Scope 9

2.4. Project Overview 11

2.5. Project Timeline 12

3. Detailed Findings **12**

3.1. NexusOFT deployed with zero token ID cannot be registered 13

3.2. Lack of zero-address validation for native-token fee recipient 15

3.3. Unchecked `uint64` downcast in `_toSD` 17

3.4. OAppCoreRBACUpgradeable does not enforce delegate synchronization with `DEFAULT_ADMIN_ROLE` at initialization 18

3.5. Both `_setBlacklist` and `_setWhitelist` can be optimized 20

3.6. After token deregistration, the fee bps settings are not reset 22

3.7. The RateLimiter does not have limitation for the limit and window 24

3.8. Nexus does not validate the result of `mint` and `burn` functions 26

3.9. OFT deregistration can block incoming token transfers 28

3.10. The `recoverFunds` function allows to transfer funds to a blocked receiver 30

4. Discussion 31

4.1. Insufficient sender balance is detected late in the send flow, wasting gas on failed transactions 32

4.2. Native tokens sent via `_lzReceive` in Nexus can be locked 32

4.3. Missing event emission for `_accrueFee` function 34

4.4. The `isPeer` function accepts zero value as a valid peer 34

4.5. Insufficient test coverage 35

5. Threat Model 36

5.1. Module: AllowlistBaseUpgradeable.sol 37

5.2. Module: AllowlistRBACUpgradeable.sol 39

5.3. Module: ERC20Plus.sol 42

5.4. Module: Nexus.sol 48

5.5. Module: NexusERC20.sol 55

5.6. Module: NexusOFT.sol 61

5.7. Module: NexusOFTAlt.sol 62

6. Assessment Results 64

6.1. Disclaimer 65

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for LayerZero Labs in several phases during 2026. The primary review ran from January 14 to February 3. After a remediation period, we performed two further review phases: on February 27 and from March 13 to March 19. This document combines those periods into a single report.

During the engagement, Zellic reviewed LayerZero Console's code for security vulnerabilities, design issues, and general weaknesses in security posture. Findings first reported during the primary review are indicated with call-out boxes in the findings sections. The threat model in this report reflects the state of the code as of the third review period.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Can an attacker bypass the Nexus registry logic to register malicious tokens or hijack existing token IDs?
- Can users bypass the validations and send tokens even if they are on the blacklist or not on the whitelist, depending on the current mode?
- Is the fee-handling mechanism in Nexus vulnerable to griefing or miscalculation exploits?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

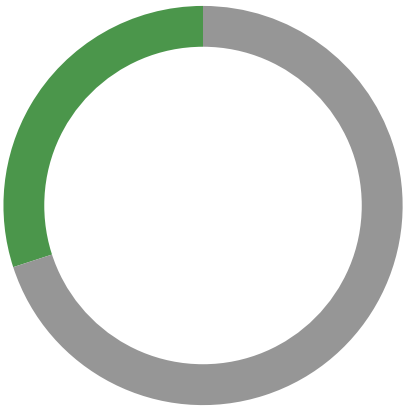
During our assessment on the scoped LayerZero Console contracts, we discovered 10 findings. No critical issues were found. Three findings were of low impact and the remaining findings were

informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of LayerZero Labs in the Discussion section (4. ↗).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	3
Informational	7



2. Introduction

2.1. About LayerZero Console

LayerZero Labs contributed the following description of LayerZero Console:

> Console contracts are regular OFTs with extensions (rate limiter, pausability, fees) to expand ERC20 tokens, also with extensions (whitelist, blacklist, pausability). Nexus contracts are a common messaging layer with which ERC20s are extended to inherit OFT capabilities, and the same extensions as above.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

LayerZero Console Contracts

Type	Solidity
Platform	EVM-compatible
Target	Primary review period
Repository	https://github.com/LayerZero-Labs/console-internal ↗
Version	`5de8a33e7db091e5c4d0f4320a851218d0d04e68`
Programs	`contracts/*`
Target	Second review period
Repository	https://github.com/LayerZero-Labs/console-audit/ ↗
Version	`79cefc35512bb2021445f0e82119c65bca579de4`

Programs	`apps/console/backend/console-oft-app/contracts/evm/contracts/*`
	`apps/oapp-app/contracts/evm/non-upgradeable/contracts/*`
	`apps/oapp-app/contracts/evm/upgradeable/contracts/*`
	`apps/oft-app/contracts/evm/non-upgradeable/contracts/*`
	`apps/oft-app/contracts/evm/upgradeable/contracts/*`
	`contracts/common/utils/evm/non-upgradeable/contracts/*`
	`contracts/common/utils/evm/upgradeable/contracts/*`

Target	Third review period
Repository	https://github.com/LayerZero-Labs/console-audit ↗
Version	`47a1dab2e790ea6e3ee838f4d99bac46b2516df2`

Programs	`apps/project-types/console-oft-app/contracts/evm/contracts/*`
	`apps/project-types/nexus-appcontracts/evm/contracts/*`
	`contracts/common/utils/evm/non-upgradeable/contracts/*`
	`contracts/common/utils/evm/upgradeable/contracts/*`

Target	Only changes with https://github.com/LayerZero-Labs/devtools/tree/306ee690/packages/oapp-evm/contracts
Repository	https://github.com/LayerZero-Labs/console-audit ↗
Version	`88f2e98b0fa9f675907ea8c53b37b7f024e32be6`
Programs	`apps/oapp-app/contracts/evm/non-upgradeable/contracts/*`

Target	Only changes with https://github.com/LayerZero-Labs/devtools/tree/306ee690/packages/oapp-evm-upgradeable/contracts
Repository	https://github.com/LayerZero-Labs/console-audit ↗
Version	`88f2e98b0fa9f675907ea8c53b37b7f024e32be6`
Programs	`apps/oapp-app/contracts/evm/upgradeable/contracts/*`

Target	Only changes with https://github.com/LayerZero-Labs/devtools/tree/306ee690/packages/oft-evm/contracts
Repository	https://github.com/LayerZero-Labs/console-audit ↗
Version	`47a1dab2e790ea6e3ee838f4d99bac46b2516df2`
Programs	`apps/oft-app/contracts/evm/non-upgradeable/contracts/*`

Target	Only changes with https://github.com/LayerZero-Labs/devtools/tree/306ee690/packages/oft-evm-upgradeable/contracts
Repository	https://github.com/LayerZero-Labs/console-audit ↗
Version	`47a1dab2e790ea6e3ee838f4d99bac46b2516df2`
Programs	`apps/oft-app/contracts/evm/upgradeable/contracts/*`

Target	Final commit
Repository	https://github.com/LayerZero-Labs/monorepo-external/ ↗
Version	`9a1436b0dad246e67dc0f44070091288dabd255c`

Programs	`apps/oft-app/*`
	`apps/oapp-app/*`
	`apps/project-types/*`
	`contracts/common/utils/*`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of two person-weeks. The assessment was conducted by two consultants over the course of one calendar week.

Contact Information

The following project manager was associated with the engagement:

Chad McDonald
Engagement Manager
chad@zellic.io

The following consultants were engaged to conduct the assessment:

Katerina Belotskaia
Engineer
kate@zellic.io

Ethan Lee
Engineer
ethl@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

January 14, 2026	Start of primary review period
February 3, 2026	End of primary review period
February 27, 2026	Start of the second review period
February 27, 2026	End of the second review period
March 13, 2026	Start of the third review period
March 19, 2026	End of the third review period

3. Detailed Findings

3.1. NexusOFT deployed with zero token ID cannot be registered

Target	NexusOFT		
Category	Code Maturity	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The NexusOFT constructor accepts a `_tokenId` parameter and stores it as an immutable variable without validating that the value is nonzero.

```
constructor(address _nexus, address _token, uint32 _tokenId) {
    NEXUS = INexus(_nexus);
    ENDPOINT = IOAppCore(_nexus).endpoint();
    INNER_TOKEN = _token;
    TOKEN_ID = _tokenId;
}
```

And `OFTRegistryBaseUpgradeable._registerToken` explicitly rejects a token ID of zero, treating it as an invalid registration:

```
function _registerToken(uint32 _tokenId, address _oftAddress,
    address _burnerMinterAddress) internal virtual {
    if (_tokenId == 0 || _burnerMinterAddress == address(0) || _oftAddress ==
    address(0)) {
        revert InvalidTokenRegistration(_tokenId, _burnerMinterAddress,
    _oftAddress);
    }
    // [...]
}
```

This restriction is consistent with how the registry uses zero as a sentinel value for unregistered OFTs. The `_getAndAssertTokenId` helper treats a return value of zero from the `oftToId` mapping as evidence that the address is not registered:

```
function _getAndAssertTokenId(address _oftAddress)
    internal view virtual returns (uint32 tokenId) {
    tokenId = getTokenId(_oftAddress);
    if (tokenId == 0) revert InvalidOFT(_oftAddress);
}
```

Because `TOKEN_ID` is immutable, a NexusOFT deployed with `_tokenId = 0` cannot be corrected after the fact; the deployment must be discarded and the contract redeployed with a valid identifier.

Impact

A NexusOFT contract mistakenly deployed with a zero token ID can never be registered in the Nexus registry. All subsequent calls to `_registerToken` for that contract will revert with `InvalidTokenRegistration`, making the deployment permanently unusable.

Recommendations

Add a nonzero check in the NexusOFT constructor:

```
constructor(address _nexus, address _token, uint32 _tokenId) {
   if (_tokenId == 0) revert InvalidTokenId(_tokenId);
    NEXUS = INexus(_nexus);
    ENDPOINT = IOAppCore(_nexus).endpoint();
    INNER_TOKEN = _token;
    TOKEN_ID = _tokenId;
}
```

Remediation

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in commit cb3b7327 ↗.

3.2. Lack of zero-address validation for native-token fee recipient

Target	PortalOFTNative		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

This finding was discovered during the primary review period.

The `_transferFees` function in PortalOFTNative withdraws a specified amount of native tokens from the contract to the recipient `_to` using a low-level `call`. However, the function does not validate that `_to` is not the zero address. Because a low-level `call` can successfully send native tokens to `address(0)`, the withdrawn fees would be irrecoverably burned.

The following snippet shows the `_transferFees` implementation.

```
function _transferFees(address /* _token */, address _to, uint256 _amount)
    internal virtual override {
    (bool success, bytes memory data) = payable(_to).call{ value: _amount
    }("");
    if (!success) {
        revert WithdrawFeesFailed(_to, _amount, data);
    }
}
```

The function only checks whether the `call` succeeded, not whether the recipient is a valid nonzero address. If `_to` is `address(0)`, the call will still succeed and the native tokens will be permanently lost.

Impact

If `_to` is set to the zero address, the transaction will succeed and the withdrawn fees will be burned.

Recommendations

We recommend adding a zero-address check for `_to` before performing the transfer.

```
function _transferFees(address /* _token */, address _to, uint256 _amount)
    internal virtual override {
   require(_to != address(0), "invalid recipient");
    (bool success, bytes memory data) = payable(_to).call{ value: _amount
    }("");
    if (!success) {
        revert WithdrawFeesFailed(_to, _amount, data);
    }
}
```

Remediation

This issue has been acknowledged by LayerZero Labs.

3.3. Unchecked `uint64` downcast in `_toSD`

Target	DecimalUtils		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

This finding was discovered during the primary review period.

The helper `_toSD` divides a local-decimal amount by `DECIMAL_CONVERSION_RATE` and then casts the result to `uint64` without any bounds checking. If the quotient exceeds `type(uint64).max`, the explicit cast silently truncates the value rather than reverting.

The following snippet shows the affected function.

```
function _toSD(uint256 _amountLD)
    internal view virtual returns (uint64 amountSD) {
    return uint64(_amountLD / DECIMAL_CONVERSION_RATE);
}
```

Impact

The converted shared-decimal amount can be silently truncated.

Recommendations

Consider adding a safe cast to ensure the `_amountLD / DECIMAL_CONVERSION_RATE` does not exceed `type(uint64).max`.

Remediation

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in commit `f5bb749d` ↗.

3.4. OAppCoreRBACUpgradeable does not enforce delegate synchronization with DEFAULT_ADMIN_ROLE at initialization

Target	OAppCoreRBACUpgradeable		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

OAppCoreRBACUpgradeable is designed around a core invariant: the LayerZero endpoint delegate is always the same address as the `DEFAULT_ADMIN_ROLE` holder. The contract enforces this post-initialization by reverting any direct call to `setDelegate` and automatically syncing the delegate whenever `acceptDefaultAdminTransfer` is called.

```
function setDelegate(address) public virtual {
    revert CannotDirectlySetDelegate();
}

function acceptDefaultAdminTransfer() public virtual override {
    super.acceptDefaultAdminTransfer();
    _setDelegate(defaultAdmin());
}
```

However, this invariant is not enforced at initialization.

The initial endpoint delegate is set through `OAppCoreBaseUpgradeable.__OAppCoreBase_init`, which accepts any address, while the initial `DEFAULT_ADMIN_ROLE` holder is set through `AccessControl2StepUpgradeable.__AccessControl2Step_init` independently. OAppCoreRBACUpgradeable does not override either function or provide a combined initializer, so the responsibility for passing the same address to both calls falls entirely on the inheriting contract.

```
// OAppCoreBaseUpgradeable — sets the endpoint delegate
function __OAppCoreBase_init_unchained(address _delegate)
    internal onlyInitializing {
    if (_delegate == address(0)) revert InvalidDelegate();
    endpoint.setDelegate(_delegate);
}

// OAppCoreRBACUpgradeable — empty; does not coordinate the two init calls
function __OAppCoreRBAC_init() internal onlyInitializing {}
```

The contract's NatSpec further omits any mention of this initialization requirement, stating only that the delegate is permanently synced without noting the condition under which this holds.

Inheriting contracts in the current codebase pair the calls correctly. For example, OFTCoreExtendedRBACUpgradeable passes the same `_initialAdmin` to both:

```
function __OFTCoreExtendedRBAC_init(address _initialAdmin,
    address _feeDeposit) internal onlyInitializing {
    __OAppCoreBase_init_unchained(_initialAdmin);
    __AccessControl2Step_init_unchained(_initialAdmin);
    // [...]
}
```

A future inheriting contract that inadvertently passes different addresses to these two functions would deploy with an endpoint delegate that differs from the `DEFAULT_ADMIN_ROLE` holder. Because `setDelegate` always reverts, the only way to correct the mismatch would be to transfer the admin role through the two-step process, which moves the delegate to the new admin.

Impact

A contract deployed with a mismatched initial delegate grants endpoint configuration privileges to an address that does not hold `DEFAULT_ADMIN_ROLE`. The admin has no direct way to revoke this; `setDelegate` is permanently blocked, and the mismatch can only be corrected indirectly through an admin transfer.

Recommendations

Provide a coordinated initializer that accepts the initial admin and ensures both the endpoint delegate and `DEFAULT_ADMIN_ROLE` are set to the same address. Inheriting contracts should call this single function rather than independently calling both base initializers.

Or update the contract NatSpec to document the requirement explicitly.

Remediation

The team updated the contract NatSpec to document explicitly that the initial endpoint delegate and the `DEFAULT_ADMIN_ROLE` holder must be the same address at initialization.

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in commit `88f2e98b` ↗.

3.5. Both `_setBlacklist` and `_setWhitelist` can be optimized

Target	AllowlistCore, AllowlistCoreUpgradeable		
Category	Business Logic	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

This finding was discovered during the primary review period.

The `_setBlacklisted` and `_setWhitelisted` functions add or remove users from the blacklist and whitelist, respectively. Before updating a list, they call `contains` to check whether the address is already present when `isEnabled == true` or absent when `isEnabled == false`, reverting if the desired state is already achieved. After that, they invoke the OpenZeppelin `EnumerableSet` `add` or `remove` functions. However, `EnumerableSet.add` and `EnumerableSet.remove` also perform a `contains` check, resulting in duplicate lookups for each operation.

```
function _setBlacklisted(SetAllowlistParam[] calldata _params)
    internal virtual {
    for (uint256 i = 0; i < _params.length; i++) {
        SetAllowlistParam calldata param = _params[i];
        if (param.isEnabled == _blacklistedSet.contains(param.user)) {
            revert AllowlistStateIdempotent(param.user, param.isEnabled);
        }
        if (param.isEnabled) {
            _blacklistedSet.add(param.user);
        } else {
            _blacklistedSet.remove(param.user);
        }
        emit BlacklistUpdated(param.user, param.isEnabled);
    }
}

function _add(Set storage set, bytes32 value) private returns (bool) {
    if (!_contains(set, value)) {
        set._values.push(value);
        // The value is stored at length-1, but we add 1 to all indexes
        // and use 0 as a sentinel value
        set._positions[value] = set._values.length;
        return true;
```

```
    } else {
        return false;
    }
}
```

Impact

Redundant `contains` checks increase gas usage.

Recommendations

Rely on the return value of `add` and `remove` functions, and revert if they return `false`.

Remediation

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in commit 127908d2 ↗.

3.6. After token deregistration, the fee bps settings are not reset

Target	OFTRegistryCoreUpgradeable		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

This finding was discovered during the primary review period.

The `deregisterToken` function in OFTRegistryCoreUpgradeable removes the token's registration from the registry mappings (`idToOft`, `idToMinterBurner`, `oftToId`) but does not clear the associated configuration states in extensions such as `FeeCoreUpgradeable`, `RateLimiterCoreUpgradeable`, and `PauseUpgradeable`.

These configurations (e.g., fees, rate limits, pause status) are keyed by the Nexus ID, which is derived from the `tokenId`. Consequently, if the owner registers a new OFT using a `tokenId` that was previously used and deregistered, the new token will inherit the old configuration values.

```
function _deregisterToken(uint32 _tokenId) internal virtual {
    OFTRegistryCoreStorage storage $ = _getOFTRegistryCoreStorage();

    address oftAddress = $.idToOft[_tokenId];

    if (oftAddress == address(0)) {
        revert TokenNotRegistered(_tokenId);
    }

    delete $.idToOft[_tokenId];
    delete $.idToMinterBurner[_tokenId];
    delete $.oftToId[oftAddress];

    emit TokenDeregistered(_tokenId);
}
```

Impact

Newly registered tokens that reuse a `tokenId` may inherit unintended configurations. Although the likelihood is low, the fee bps may be inadvertently applied to a new OFT that utilizes a `tokenId`

previously assigned to another OFT after its deregistration. Since both registration and configuration are controlled by the owner, this issue primarily presents a risk of misconfiguration rather than a malicious exploit.

Recommendations

We recommend to update `deregisterToken` to explicitly clear or reset the fee, rate limit, and pause states associated with the `tokenId`.

Remediation

This issue has been acknowledged by LayerZero Labs.

LayerZero Labs provided the following response to this finding:

> Token IDs are not meant to be reused for different tokens, it's ok if extensions keep the state.

3.7. The RateLimiter does not have limitation for the limit and window

Target	RateLimiterCoreUpgradeable		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

This finding was discovered during the primary review period.

The RateLimiterCoreUpgradeable contract allows the configuration of rate limits via the `_setRateLimitConfigs` function. This function updates the rate-limit parameters, including `outboundLimit`, `inboundLimit`, `outboundWindow`, and `inboundWindow`, based on the input `SetRateLimitConfigParam`.

However, the contract does not enforce any minimum values for the limit or maximum values for the window. While the `Math.saturatingSub` prevents underflow, extreme configurations are permissible.

If the limit is extremely small and the window is extremely large, the decay rate becomes negligible.

```
function _setRateLimitConfigs(SetRateLimitConfigParam[] calldata _params)
    internal virtual {
    RateLimiterCoreStorage storage $ = _getRateLimiterCoreStorage();

    for (uint256 i = 0; i < _params.length; i++) {
        // [...]

        // does not have any limitation.
        rateLimit.outboundLimit = param.config.outboundLimit;
        rateLimit.inboundLimit = param.config.inboundLimit;
        rateLimit.outboundWindow = param.config.outboundWindow;
        rateLimit.inboundWindow = param.config.inboundWindow;

        emit RateLimitConfigUpdated(param.id, param.config);
    }
}
```

Impact

The impact is primarily operational and depends on the intent of the `RATE_LIMITER_ADMIN_ROLE`. The Nexus contract supports pausing sending but does not natively support pausing receiving. However, an owner can effectively achieve a receive pause by configuring a rate limit with a limit of zero and a very large window.

Recommendations

We recommend to introduce sanity checks in `_setRateLimitConfigs` to prevent limits from being set to zero or windows from being excessively large.

Remediation

This issue has been acknowledged by LayerZero Labs.

3.8. Nexus does not validate the result of `mint` and `burn` functions

Target	Nexus, PortalOFTMintBurn		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

This finding was discovered during the primary review period.

The `mint` and `burn` functions should return the success status of the operation. However, the following functions ignore these results.

- `Nexus._debit`
- `Nexus._credit`
- `PortalOFTMintBurn._debit`
- `PortalOFTMintBurn._credit`

The following snippet illustrates the pattern in `Nexus._debit`, where both `burn` and `mint` are called without inspecting their return values.

```
function _debit(
    // [...]
) internal virtual whenNotPaused(_nexusId) returns (uint256 amountSentLD,
    uint256 amountReceivedLD) {
    // [...]

    // Does not check burn's return value.
    IMintableBurnable(_minterBurner).burn(_from, amountSentLD);

    if (amountSentLD > amountReceivedLD) {
        unchecked {
            // Does not check mint's return value.
            IMintableBurnable(_minterBurner).mint(address(this), amountSentLD
    - amountReceivedLD);
        }
    }
}
```

Impact

As of the time of writing, both the Nexus and Portal ERC20 contracts trigger a revert upon a `mint` or `burn` failure; therefore, there is no immediate impact, and the finding has been classified as Informational. However, this may be a significant factor in future implementations, if the IMintableBurnable `mint` and `burn` implementation returns a success result instead of a revert, which will be skipped.

Recommendations

We recommend to verify the return values of the `mint` and `burn` operations.

Remediation

This issue has been acknowledged by LayerZero Labs.

3.9. OFT deregistration can block incoming token transfers

Target	OFTRegistryUpgradeable		
Category	Business Logic	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

This finding was discovered during the primary review period.

The OFTRegistryUpgradeable contract supports the `deregisterToken` functionality, which allows the contract owner to remove a previously registered OFT. Deregistration prevents that token from interacting with the Nexus contract. As a result, any in-flight cross-chain transfers for the token will be halted and cannot be completed.

Impact

If cross-chain transfers were not completed before the token was deregistered, they cannot be finalized afterward.

Recommendations

Consider adding safeguards around deregistration — for example, require the system to be paused, and ensure no pending transfers exist for the token.

Remediation

This issue has been acknowledged by LayerZero Labs.

LayerZero Labs provided the following response to this finding:

> Token de-registration is equivalent to peer setting, so it's expected that behaves the same way for inflight messages.
>
> We can work with extensions to handle specific token un-wiring so that we first pause outgoing transfers (rate limiter to 0, all pathways paused, token pause, token blacklist from Nexus, etc.), deliver all inflight messages as incoming transfers are still allowed, and then de-register the

token.

3.10. The `recoverFunds` function allows to transfer funds to a blocked receiver

Target	PortalERC20, NexusERC20		
Category	Business Logic	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

This finding was discovered during the primary review period.

The NexusERC20 and PortalERC20 tokens support the allowlist, which can either fully block interactions for blacklisted users or restrict interactions to whitelisted users. Additionally, callers with the `FUND_RECOVERY_ROLE` can transfer funds from currently blocked users using the `recoverFunds` function. However, this function does not validate that the recipient is also allowlisted.

Below is the `recoverFunds` function implementation from the NexusERC20 contract

```
function recoverFunds(address _from, address _to, uint256 _amount)
    public virtual onlyRole(FUND_RECOVERY_ROLE) {
    if (_getNexusERC20Storage().guard.isAllowlisted(_from))
    revert CannotRecoverFromAllowlisted(_from);
    super._transfer(_from, _to, _amount);
}
```

and from the PortalERC20 contract:

```
function recoverFunds(address _from, address _to, uint256 _amount)
    public virtual onlyRole(FUND_RECOVERY_ROLE) {
    if (isAllowlisted(_from)) revert CannotRecoverFromAllowlisted(_from);
    super._transfer(_from, _to, _amount);
}
```

Impact

Funds can be recovered to an account that is not currently allowlisted, leaving those tokens blocked.

Recommendations

Consider validating that the specified recipient address is allowlisted.

```
function recoverFunds(address _from, address _to, uint256 _amount)
    public virtual onlyRole(FUND_RECOVERY_ROLE) {
    if (isAllowlisted(_from)) revert CannotRecoverFromAllowlisted(_from);
    if (!isAllowlisted(_to)) revert RecepientIsNotAllowlisted(_to);
    super._transfer(_from, _to, _amount);
}

function recoverFunds(address _from, address _to, uint256 _amount)
    public virtual onlyRole(FUND_RECOVERY_ROLE) {
    if (_getNexusERC20Storage().guard.isAllowlisted(_from))
    revert CannotRecoverFromAllowlisted(_from);

    if (!_getNexusERC20Storage().guard.isAllowlisted(_to)) revert RecepientIsNo
       tAllowlisted(_to);
    super._transfer(_from, _to, _amount);
}
```

Remediation

This issue has been acknowledged by LayerZero Labs.

LayerZero Labs provided the following response to this finding:

> It's intended to be a force transfer function with some flexibility.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Insufficient sender balance is detected late in the send flow, wasting gas on failed transactions

When a user initiates a cross-chain transfer through NexusOFT, the send flow passes through several stateful operations before the sender's token balance is verified. In `Nexus._debit`, the execution order is as follows: pause check, fee calculation with slippage validation, rate-limiter outflow, and finally the burn call that implicitly validates the balance.

```
function _debit(
    uint256 _nexusId,
    address _burnerMinter,
    address _from,
    uint256 _amountLD,
    uint256 _minAmountLD
) internal virtual returns (uint256 amountSentLD, uint256 amountReceivedLD) {
    if (_isPaused(_nexusId)) revert INexusPause.Paused(_nexusId);

    (amountSentLD, amountReceivedLD) = _debitView(_nexusId, _amountLD,
    _minAmountLD);

    _outflow(_nexusId, _from, amountReceivedLD);  // Rate limiter state write

    _burnAndCollectFee(_burnerMinter, _from, amountSentLD,
    amountReceivedLD);  // Balance validated here
}
```

The burn call inside `_burnAndCollectFee` is the only place where the sender's balance is implicitly checked, and it is the last operation in `_debit`. If the sender does not hold enough tokens, the entire transaction reverts, correctly rolling back the rate-limiter state change. Correctness is not affected. However, a user who submits a transaction with an insufficient balance will consume all gas until the transaction is reverted.

Consider adding an explicit balance check at the `NexusOFT.send()` function.

4.2. Native tokens sent via `_lzReceive` in Nexus can be locked

The function `Nexus._lzReceive` does not check `msg.value` to revert when it is nonzero.

```
// apps/project-types/nexus-app/contracts/evm/contracts/Nexus.sol
function _lzReceive(
    Origin calldata _origin,
    bytes32 _guid,
    bytes calldata _message,
    address /* _executor */,
    bytes calldata /* _extraData */
) internal virtual override {
    uint32 tokenId = _message.tokenId();
    address oftAddress = _getAndAssertOFTAddress(tokenId);
    // [...]
}
```

Since `OAppReceiver.lzReceive` is declared `payable`, the executor can forward native tokens to the OFT or Nexus contract on the destination chain.

```
// LayerZero-v2/packages/layerzero-v2/evm/oapp/contracts/oapp/OAppReceiver.sol
function lzReceive(
    Origin calldata _origin,
    bytes32 _guid,
    bytes calldata _message,
    address _executor,
    bytes calldata _extraData
) public payable virtual {
    if (address(endpoint) != msg.sender) revert OnlyEndpoint(msg.sender);
    if (_getPeerOrRevert(_origin.srcEid) != _origin.sender)
    revert OnlyPeer(_origin.srcEid, _origin.sender);
    _lzReceive(_origin, _guid, _message, _executor, _extraData);
}
```

A user can encode a nonzero native value into message options using `addExecutorLzReceiveOption`.

```
// LayerZero-v2/packages/layerzero-
    v2/evm/oapp/contracts/oapp/libs/OptionsBuilder.sol
function addExecutorLzReceiveOption(
    bytes memory _options,
    uint128 _gas,
    uint128 _value
) internal pure onlyType3(_options) returns (bytes memory) {
    bytes memory option = ExecutorOptions.encodeLzReceiveOption(_gas, _value);
    return addExecutorOption(_options, ExecutorOptions.OPTION_TYPE_LZRECEIVE,
    option);
}
```

When this option specifies a nonzero `_value`, the executor collects that amount from the sender on the source chain as part of the LayerZero fee and forwards it as `msg.value` when calling `lzReceive` on the destination. Neither the OFT nor Nexus `_lzReceive` implementations consume `msg.value`, and neither contains a native withdrawal mechanism. Any native value forwarded in this way is therefore locked in the contract.

We suggested that the team should add checks to revert when `msg.value` is nonzero.

The LayerZero Labs team acknowledged this behavior.

4.3. Missing event emission for `_accrueFee` function

This discussion point was discovered during the primary review period.

The `_accrueFee` function modifies state but does not emit an event. This can make it difficult to track fee accounting off chain.

```
function _accrueFee(address _token, uint256 _amount) internal virtual {
    FeeCoreStorage storage $ = _getFeeCoreStorage();
    unchecked {
        $.feeBalances[_token] += _amount;
    }
}
```

Consider adding an event for the `_accrueFee` function.

4.4. The `isPeer` function accepts zero value as a valid peer

This discussion point was discovered during the primary review period.

The `isPeer` function returns the result of a raw equality check between the stored mapping value and the supplied `_peer` (`peers(_eid) == _peer`). Because Solidity mappings return the zero value for unset keys, calling `isPeer(_eid, bytes32(0))` will return true whenever `peers[_eid]` is unset.

Any higher-level logic that treats a true response from `isPeer` as proof that a peer relationship was explicitly configured will therefore accept an uninitialized or cleared entry.

Consider adding a check that the provided `_peer` is not zero.

```
function isPeer(
    uint32 _eid,
    bytes32 _peer
)
    public
    view
    virtual
    override(IOAppPreCrimeSimulator, OAppPreCrimeSimulatorUpgradeable)
    returns (bool isValidPeer)
{
    return peers(_eid) == _peer;
}
```

4.5. Insufficient test coverage

This discussion point was discovered during the primary review period.

When building a complex contract with multiple moving parts (state changes) and dependencies, comprehensive testing is essential. This includes testing for both positive and negative scenarios on every function that touches the contract's state.

Positive tests should verify that each function's side effect is as expected, while negative tests should cover every revert, preferably in every logical branch.

The test coverage for this project should be expanded to include all contracts and all functions — not just surface-level functions. It is important to test the invariants required for ensuring security and also verify any mathematical properties from the project's specification. Additionally, testing cross-chain function calls and transfers is recommended to ensure the desired functionality.

Good test coverage has multiple effects:

- It finds bugs and design flaws early (preaudit or prerelease).
- It gives insight into areas for optimization (e.g., gas cost).
- It displays code maturity.
- It bolsters customer trust in the product.
- It improves understanding of how the code functions, integrates, and operates — for developers and auditors alike.
- It increases development velocity long-term.

The last point may seem contradictory given the time investment to create and maintain tests. However, tests help developers trust their own changes. It is difficult to know if a code refactor — or

even just a small one-line fix — breaks other code if there are no tests. This is especially true for new developers or those returning to the code after a prolonged absence.

Tests have your back here. They are an excellent indicator that the existing functionality was most likely not broken by a change to the code. Without a comprehensive test suite, the above benefits are lost. This increases the likelihood of bugs and vulnerabilities going unnoticed until after deployment, which can be costly and damaging to the project's reputation.

We recommend expanding the test suite to ensure extensive and thorough test coverage.

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

The current threat model reflects the state of the code as of the third review period.

5.1. Module: AllowlistBaseUpgradeable.sol

Function: `isAllowlisted(address _user)`

This is a view function that returns true if the user is allowlisted. The result depends on the current mode:

- If the mode is Open, it should return true for all users.
- If the mode is Blacklist, it should return false for users in the blacklist and true for users not in the blacklist. The whitelist is ignored in this case.
- If the mode is Whitelist, it should return true for users in the whitelist and false for users not in the whitelist. The blacklist is ignored in this case.

The code of the function does not restrict the case when the same user is added to the blacklist and whitelist, and it does not affect the result of the function.

Inputs

- `_user`
  - **Control**: Full control by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The user address to check.

Branches and code coverage

Intended branches

- When mode is Open, return true for all users.
  - ☑ Test coverage
- When mode is Blacklist, return false for users in the blacklist and true for users not in the blacklist. The whitelist is ignored in this case.
  - ☑ Test coverage
- When mode is Whitelist, return true for users in the whitelist and false for users not in

the whitelist. The blacklist is ignored in this case.

- [x] Test coverage

Function: `isBlacklisted(address _user)`

This is a view function that returns true if the user is blacklisted.

Inputs

- `_user`
  - **Control**: Full control by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The user address to check.

Branches and code coverage

Intended branches

- Return true if the user is blacklisted. The mode is not taken into account.
  - [x] Test coverage
- Return false if the user is not blacklisted. The mode is not taken into account.
  - [ ] Test coverage

Function: `isWhitelisted(address _user)`

This is a view function that returns true if the user is whitelisted.

Inputs

- `_user`
  - **Control**: Full control by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The user address to check.

Branches and code coverage

Intended branches

- Return true if the user is whitelisted. The mode is not taken into account.
  - ☑ Test coverage
- Return false if the user is not whitelisted. The mode is not taken into account.
  - ☑ Test coverage

5.2. Module: AllowlistRBACUpgradeable.sol

Function: `setAllowlistMode(AllowlistMode _mode)`

This function allows the caller with a `DEFAULT_ADMIN_ROLE` role to set the allowlist mode. The mode can be set to Open, Blacklist, or Whitelist.

Inputs

- `_mode`
  - **Control**: Full control by the admin.
  - **Constraints**: Cannot be set to the same mode as the current one.
  - **Impact**: The new mode to set.

Branches and code coverage

Intended branches

- Successfully set the new mode.
  - ☑ Test coverage
- Successful multiple mode updates.
  - ☑ Test coverage
- When the mode is set to Open, all users are allowed.
  - ☑ Test coverage
- When the mode is set to Blacklist, users in the blacklist are not allowed. The whitelist is ignored in this case.
  - ☑ Test coverage
- When the mode is set to Whitelist, users in the whitelist are allowed. The blacklist is ignored in this case.
  - ☑ Test coverage

Negative behavior

- Revert if the caller does not have the `DEFAULT_ADMIN_ROLE` role.

- ☑ Negative test
- Revert if the new mode is equal to the current one.
  - ☑ Negative test

Function: `setBlacklisted(SetAllowlistParam[] _params)`

This function allows the caller with a `BLACKLISTER_ROLE` role to add or remove users from the blacklist. The `SetAllowlistParam` contains the user address and the blacklist state.

Inputs

- `_params`
  - **Control**: Full control by the `BLACKLISTER_ROLE` role.
  - **Constraints**: Revert if an already blacklisted user should be added to the blacklist.
  - **Impact**: The list of users' addresses and their blacklist states.

Branches and code coverage

Intended branches

- The blacklist has been updated successfully while adding a new user to the blacklist.
  - ☑ Test coverage
- The blacklist has been updated successfully while removing a user from the blacklist.
  - ☑ Test coverage
- Successfully update the blacklist for multiple users.
  - ☑ Test coverage
- No side effects on the whitelist.
  - ☑ Test coverage
- If the mode is Blacklist, `isAllowlisted` returns false for the blacklisted users.
  - ☑ Test coverage
- The Open or Whitelist mode is not affected by the blacklist update.
  - ☑ Test coverage
- Successful execution if the blacklist is empty.
  - ☑ Test coverage

Negative behavior

- The caller is not a `BLACKLISTER_ROLE` role.
  - ☑ Negative test
- Revert if an already blacklisted user should be added.
  - ☑ Negative test
- Revert if a not-blacklisted user should be removed.
  - ☑ Negative test
- Revert for all users even if one element is idempotent.
  - ☑ Negative test

Function: `setWhitelisted(SetAllowlistParam[] _params)`

This function allows the caller with a `WHITELISTER_ROLE` role to add or remove users from the whitelist. The `SetAllowlistParam` contains the user address and the whitelist state.

Inputs

- `_params`
  - **Control**: Full control by the `WHITELISTER_ROLE` role.
  - **Constraints**: Revert if an already whitelisted user should be added to the whitelist.
  - **Impact**: The list of users' addresses and their whitelist states.

Branches and code coverage

Intended branches

- The whitelist has been updated successfully while adding a new user to the whitelist.
  - ☑ Test coverage
- The whitelist has been updated successfully while removing a user from the whitelist.
  - ☑ Test coverage
- Successfully updates the whitelist for multiple users.
  - ☑ Test coverage
- No side effects on the blacklist.
  - ☑ Test coverage
- If the mode is Whitelist, `isAllowlisted` returns true for the whitelisted users.
  - ☑ Test coverage

- The Open or Blacklist mode is not affected by the whitelist update.
  - ☑ Test coverage
- Successful execution if the whitelist is empty.
  - ☑ Test coverage

Negative behavior

- The caller is not a `WHITELISTER_ROLE` role.
  - ☑ Negative test
- Revert if an already whitelisted user should be added.
  - ☑ Negative test
- Revert if a not-whitelisted user should be removed.
  - ☑ Negative test
- Revert for all users even if one element is idempotent.
  - ☑ Negative test

5.3. Module: ERC20Plus.sol

Function: `burn(address _from, uint256 _amount)`

This function allows a caller with the burner role to destroy tokens from a specified address, subject to pause status and allowlist verification.

Inputs

- `_from`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must be allowlisted and have at least `_amount` tokens.
  - **Impact**: The address from which tokens are burned.
- `_amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.
  - **Impact**: The quantity of tokens to be burned.

Branches and code coverage

Intended branches

- Check if the caller has the burner role.
  - ☑ Test coverage
- Check if the contract is not globally paused.
  - ☑ Test coverage
- Check if the from address is allowlisted.
  - ☑ Test coverage
- Burn the specified amount of tokens from the address.
  - ☑ Test coverage
- Return true to indicate success.
  - ☑ Test coverage

Negative behavior

- Revert if the caller does not have the burner role.
  - ☑ Negative test
- Revert if the contract is paused.
  - ☑ Negative test
- Revert if the from address is not allowlisted.
  - ☑ Negative test
- Revert if the from address has insufficient balance.
  - ☑ Negative test

Function: `mint(address _to, uint256 _amount)`

This function allows a caller with the minter role to create new tokens and assign them to a specified address.

Inputs

- `_to`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must not be the zero address.
  - **Impact**: The recipient of the minted tokens.
- `_amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.
  - **Impact**: The quantity of tokens to be minted.

Branches and code coverage

Intended branches

- Check if the caller has the minter role.
  - ☑ Test coverage
- Mint the specified amount of tokens to the recipient.
  - ☑ Test coverage
- Return true to indicate success.
  - ☑ Test coverage

Negative behavior

- Revert if the caller does not have the minter role.
  - ☑ Negative test
- Revert if the recipient address is the zero address.
  - ☑ Negative test

Function: `recoverFunds(address _from, address _to, uint256 _amount)`

This function allows a designated recovery admin to seize and transfer funds from a nonallowlisted address to a target address.

Inputs

- `_from`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must *not* be allowlisted and must have a balance of at least `_amount`.
  - **Impact**: The address from which funds are recovered.
- `_to`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must not be the zero address.
  - **Impact**: The recipient of the recovered funds.
- `_amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.
  - **Impact**: The quantity of tokens to be recovered.

Branches and code coverage

Intended branches

- Check if the caller has the `DEFAULT_ADMIN_ROLE`.
  - ☑ Test coverage
- Verify that the `_from` address is not allowlisted.
  - ☑ Test coverage
- Transfer the specified amount from the source to the recipient.
  - ☑ Test coverage

Negative behavior

- Revert if the caller does not have the `DEFAULT_ADMIN_ROLE`.
  - ☑ Negative test
- Revert if the `_from` address is allowlisted.
  - ☑ Negative test
- Revert if the `_from` address has insufficient balance.
  - ☑ Negative test

Function: `transfer(address _to, uint256 _amount)`

This function moves tokens from the caller's account to a recipient, subject to pause status and allowlist verification for both the sender and the recipient.

Inputs

- `_to`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must be allowlisted and not be the zero address.
  - **Impact**: The recipient of the tokens.
- `_amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The caller must have a balance of at least `_amount`.
  - **Impact**: The quantity of tokens to be transferred.

Branches and code coverage

Intended branches

- Check if the contract is not paused.
  - ☑ Test coverage
- Check if the sender is allowlisted.
  - ☑ Test coverage
- Check if the recipient is allowlisted.
  - ☑ Test coverage
- Transfer the tokens from the caller to the recipient using the parent implementation.
  - ☑ Test coverage
- Return true to indicate success.
  - ☑ Test coverage

Negative behavior

- Revert if the contract is globally paused.
  - ☑ Negative test
- Revert if the sender is not allowlisted.
  - ☑ Negative test
- Revert if the recipient is not allowlisted.
  - ☑ Negative test
- Revert if the recipient is the zero address.
  - ☑ Negative test
- Revert if the caller has insufficient balance.
  - ☑ Negative test

Function: `transferFrom(address _from, address _to, uint256 _amount)`

This function moves tokens from a sender to a recipient using the allowance mechanism, subject to pause status and allowlist verification for all parties involved — sender, recipient, and caller.

Inputs

- `_from`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must be allowlisted and have a balance of at least `_amount`.
  - **Impact**: The address from which tokens are debited.
- `_to`

    - **Control**: Fully controlled by the caller.
    - **Constraints**: The address must be allowlisted and not be the zero address.
    - **Impact**: The recipient of the tokens.
- `_amount`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: The caller must have allowance from `_from` of at least `_amount`.
    - **Impact**: The quantity of tokens to be transferred.

Branches and code coverage

Intended branches

- Check if the contract is not paused.
    - ☑ Test coverage
- Check if the caller is allowlisted.
    - ☑ Test coverage
- Check if the sender is allowlisted.
    - ☑ Test coverage
- Check if the recipient is allowlisted.
    - ☑ Test coverage
- Check and update the caller's allowance from the sender.
    - ☑ Test coverage
- Transfer the tokens from the sender to the recipient using the parent implementation.
    - ☑ Test coverage
- Return true to indicate success.
    - ☑ Test coverage

Negative behavior

- Revert if the contract is paused.
    - ☑ Negative test
- Revert if the caller is not allowlisted.
    - ☑ Negative test
- Revert if the sender is not allowlisted.
    - ☑ Negative test
- Revert if the recipient is not allowlisted.
    - ☑ Negative test

- Revert if the recipient is the zero address.
  - ☑ Negative test
- Revert if the sender has insufficient balance.
  - ☑ Negative test
- Revert if the caller has insufficient allowance.
  - ☑ Negative test

5.4. Module: Nexus.sol

Function: `lzReceive(Origin _origin, byte[32] _guid, bytes _message, address, bytes)`

This function can be executed only by the trusted endpoint to deliver OFT tokens.

Inputs

- `_origin`
  - **Control**: Cross-chain messages should be saved before by the executor so only verified messages can be delivered. The caller is not actually controlling the message data.
  - **Constraints**: If the registered peer address for the `_origin.srcEid` is not the same as the `_origin.sender`, it will revert.
  - **Impact**: Contains the source endpoint ID, sender address, and nonce of the received message.
- `_guid`
  - **Control**: Cross-chain messages should be saved before by the executor so only verified messages can be delivered. The caller is not actually controlling the message data.
  - **Constraints**: No constraints.
  - **Impact**: It is used for event emission and saving a composed message.
- `_message`
  - **Control**: Cross-chain messages should be saved before by the executor so only verified messages can be delivered. The caller is not actually controlling the message data.
  - **Constraints**: No constraints.
  - **Impact**: Contains the receiver address and amount. It can also contain the composed message and sender of the composed message.
- `_executor`
  - **Control**: Full control by the caller of the `endpoint.lzReceive` function.

    - **Constraints**: No constraints.
    - **Impact**: Ignored.
- `_extraData`
    - **Control**: Full control by the caller of the `endpoint.lzReceive` function.
    - **Constraints**: No constraints.
    - **Impact**: Ignored.

Branches and code coverage

Intended branches

- The tokens have been successfully transferred to the recipient.
    - ☑ Test coverage
- A composed message has been saved.
    - ☐ Test coverage

Negative behavior

- The caller is not a trusted endpoint.
    - ☐ Negative test
- The `srcEid` is not registered.
    - ☐ Negative test
- The sender is not the expected peer for the source chain.
    - ☐ Negative test
- The receiver is zero address.
    - ☐ Negative test
- The inbound limit is reached.
    - ☐ Negative test

Function call analysis

- `this._getPeerOrRevert(_origin.srcEid)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** Returns the address of the trusted endpoint on the source chain. If this address does not match the `_origin.sender`, `lzReceive` will revert.
    - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.

- `this._lzReceive(_origin, _guid, _message, _executor, _extraData) -> this._credit(toAddress, _toLD(_message.amountSD()), _origin.srcEid) -> _inflow(_srcEid, _to, _amountLD)`
    - **What is controllable?** `_srcEid`, `_to`, and `_amountLD`.
    - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.
    - **What happens if it reverts, reenters or does other unusual control flow?** Reverts if the inbound limit is reached.
- `this._lzReceive(_origin, _guid, _message, _executor, _extraData) -> this._credit(toAddress, _toLD(_message.amountSD()), _origin.srcEid) -> _mint(_burnerMinter, _to, _amountLD)`
    - **What is controllable?** `_srcEid`, `_to`, and `_amountLD`.
    - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.
    - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- `INexusOFT(oftAddress).nexusReceive(address(endpoint), _guid, _origin.srcEid, _message.sendTo().bytes32ToAddress(), amountReceivedLD, composeMsg);`
    - **What is controllable?** `_message.sendTo`.
    - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.
    - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.

Function: `nexusSend(address _from, SendParam _sendParam, MessagingFee _fee, address _refundAddress)`

This function should be called only by the registered OFT contract. The `_getAndAssertTokenId` function will check if the caller is actually registered.

```
struct SendParam {
    uint32 dstEid; // Destination endpoint ID.
    bytes32 to; // Recipient address.
    uint256 amountLD; // Amount to send in local decimals.
    uint256 minAmountLD; // Minimum amount to send in local decimals.
    bytes extraOptions; // Additional options supplied by the caller to be used
    in the LayerZero message.
    bytes composeMsg; // The composed message for the send() operation.
    bytes oftCmd; // The OFT command to be executed, unused in default OFT
    implementations.
}
```

Inputs

- `_from`
  - **Control**: Controlled by the caller. The caller in this case is the registered OFT contract. The current Nexus OFT specifies the from address as a `msg.sender`, so it cannot be anyone except the original sender of the message.
  - **Constraints**: Should own the underlying token.
  - **Impact**: The initiator of the send operation. The underlying token will be burned from this address.
- `_sendParam`
  - **Control**: Controlled by the caller. But in the case of the NexusOFT contract, this data is directly passed from the user input.
  - **Constraints**: The `dstEid` should be trusted and registered.
  - **Impact**: Contains the `dstEid`, recipient address, amount to send, minimum amount to send, extra options, composed message, and OFT command.
- `_fee`
  - **Control**: Controlled by the caller. But in the case of the NexusOFT contract, this data is directly passed from the user input.
  - **Constraints**: The user should provide enough fee.
  - **Impact**: Contains information about the fee specified by the user. The fee will be paid to the LayerZero endpoint. If `_fee.lzTokenFee` is greater than zero, `NexusOFT.send` must have already transferred these tokens to the endpoint.
- `_refundAddress`
  - **Control**: Controlled by the caller. But in the case of the NexusOFT contract, this data is directly passed from the user input.
  - **Constraints**: `Endpoint.send` reverts if the `_refundAddress` is zero address.
  - **Impact**: The receiver of the excess fee.

Branches and code coverage

Intended branches

- The message is sent successfully.
  - ☑ Test coverage
- The return `msgReceipt` and `oftReceipt` data equal the expected values.
  - ☐ Test coverage
- The fee is default for the `nexusId`. The fee amount is equal to the expected value.
  - ☑ Test coverage
- The fee is specific for the `nexusId`. The fee amount is equal to the expected value.

- ☐ Test coverage
- Also, `Msg.inspector` is set and the message is inspected successfully.
  - ☐ Test coverage

Negative behavior

- The caller is not a registered OFT contract.
  - ☐ Negative test
- The provided fee is not enough for sending the message.
  - ☐ Negative test
- The `dstId` is not supported.
  - ☐ Negative test
- The `_nexusId` is paused.
  - ☐ Negative test
- The user does not have enough tokens to burn.
  - ☐ Negative test

Function call analysis

- `this._getAndAssertTokenId(msg.sender) -> this.getTokenId(_oftAddress)`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns the `tokenId` for the given OFT address. If the OFT is not registered, the function will revert.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts if the OFT is not registered and `tokenId` is zero.
- `this._debit(this.getNexusId(tokenId, _sendParam.dstEid), this.getMinterBurnerAddress(tokenId), _from, _sendParam.amountLD, _sendParam.minAmountLD) -> this._debitView(_nexusId, _amountLD, _minAmountLD) -> this._getFee(_nexusId, _amountLD) -> feeConfigModule_.getFee(_id, _amount)`
  - **What is controllable?** `_id`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns the fee bps for the given `_nexusId`, which contains the `tokenId` and `dstEid`. There is a global config, config for the `tokenId`, for destination EID, and for `nexusId`. The config with `MAX_PRIORITY` or with the highest priority will be returned.
  - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- `this._debit(this.getNexusId(tokenId, _sendParam.dstEid),`

`this.getBurnerMinterAddress(tokenId), _from, _sendParam.amountLD, _sendParam.minAmountLD) -> this._debitView(_nexusId, _amountLD, _minAmountLD) -> this._getFee(_nexusId, _amountLD)`

  - **What is controllable?** `_nexusId` and `_amountLD`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Calculate the fee amount using the given `_amountLD` and the result of the `_getFeeBps` function from the config module.
  - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.

- `this._debit(this.getNexusId(tokenId, _sendParam.dstEid), this.getBurnerMinterAddress(tokenId), _from, _sendParam.amountLD, _sendParam.minAmountLD) -> this._outflow(_nexusId, _from, amountReceivedLD) -> rateLimiterModule_.outflow(_id, _from, _amount)`

  - **What is controllable?** `_id` and `_amount`.
  - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** If `isGloballyDisabledFlag` is true, or if `addressExemptionEnabled` is true for the given `_id` and the `_from` address is exempted from rate limiting, the function will return early without any changes. Reverts if the `forwardAvailableAmount` is less than the `_amount`.

- `this._debit(this.getNexusId(tokenId, _sendParam.dstEid), this.getBurnerMinterAddress(tokenId), _from, _sendParam.amountLD, _sendParam.minAmountLD) -> this._burnAndCollectFee(_burnerMinter, _from, amountSentLD, amountReceivedLD) -> this._callBurnerMinter(_burnerMinter, abi.encodeWithSelector(this.BURN_SELECTOR, _from, _amountSentLD))`

  - **What is controllable?** `amountSentLD`.
  - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** Depends on the implementation of the IMintableBurnable contract. In the case of the NexusERC20 contract, it will check if `from` is allowlisted. If not, it will revert. Since the receiver is zero in the case of burning, the receiver check will be skipped. Also reverts if the balance of the from address is not enough to burn the `amountSentLD`.

- `this.getBurnerMinterAddress(tokenId)`

  - **What is controllable?** `tokenId`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns the MinterBurner address for the given `tokenId`. If the `tokenId` is not registered, the function will revert.
  - **What happens if it reverts, reenters or does other unusual control flow?** There is no check that the address is not zero, but since there is a check that

the OFT is registered, it is guaranteed that the address is not zero.

- `this._debit(this.getNexusId(tokenId, _sendParam.dstEid), this.getBurnerMinterAddress(tokenId), _from, _sendParam.amountLD, _sendParam.minAmountLD) -> this._burnAndCollectFee(_burnerMinter, _from, amountSentLD, amountReceivedLD) -> this._mint(_burnerMinter, feeDeposit(), _amountSentLD - _amountReceivedLD) -> this._callBurnerMinter(_burnerMinter, abi.encodeWithSelector(MINT_SELECTOR, _to, _amount))`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- `this._buildMsgAndOptions(_from, tokenId, _sendParam, amountReceivedLD) -> this.msgInspector()`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns the address of the message inspector, if set by the owner of the contract.
  - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- `this._buildMsgAndOptions(_from, tokenId, _sendParam, amountReceivedLD) -> IMsgInspector(inspector).inspect(_from, message, options)`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** Depends on the implementation of the IMsgInspector contract.
- `this._lzSend(_sendParam.dstEid, message, options, _fee, _refundAddress) -> this._payNative(_fee.nativeFee)`
  - **What is controllable?** `_fee.nativeFee`.
  - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** Compare the `_fee.nativeFee` with the `msg.value`. If they are not equal, it will revert.
- `this._lzSend(_sendParam.dstEid, message, options, _fee, _refundAddress) -> this._payLzToken(_fee.lzTokenFee)`
  - **What is controllable?** `_fee.lzTokenFee`.
  - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.

    - **What happens if it reverts, reenters or does other unusual control flow?** This function is overridden here and does nothing.
- `this._lzSend(_sendParam.dstEid, message, options, _fee, _refundAddress) -> this._getPeerOrRevert(_dstEid)`
    - **What is controllable?** `_dstEid`.
    - **If the return value is controllable, how is it used and how can it go wrong?** Returns the trusted peer address for the given `_dstEid` or reverts if the `_dstEid` is not registered.
    - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- `this._lzSend(_sendParam.dstEid, message, options, _fee, _refundAddress) -> this.endpoint.send{value: messageValue}(MessagingParams(_dstEid, this._getPeerOrRevert(_dstEid), _message, _options, _fee.lzTokenFee > 0), _refundAddress)`
    - **What is controllable?** `_sendParam.dstEid`, `_fee`, and `_refundAddress`.
    - **If the return value is controllable, how is it used and how can it go wrong?** Returns the `msgReceipt` data.
    - **What happens if it reverts, reenters or does other unusual control flow?** The endpoint reverts if the provided fee is not enough to pay for the message. Reverts if the LZ token is not supported but the user tries to pay the fee in LZ token.

5.5. Module: NexusERC20.sol

Function: `burn(address _from, uint256 _amount)`

This function allows a caller with the burner role to burn tokens from a specified address, bypassing the need for an allowance but subject to guard validation.

Inputs

- `_from`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: The address must have at least `_amount` tokens.
    - **Impact**: The address from which tokens are burned.
- `_amount`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: N/A.
    - **Impact**: The quantity of tokens to be burned.

Branches and code coverage

Intended branches

- Check if the caller has the burner role.
  - ☑ Test coverage
- Verify the transfer with the guard contract.
  - ☑ Test coverage
- Burn the specified amount of tokens from the address.
  - ☑ Test coverage
- Return true to indicate success.
  - ☑ Test coverage

Negative behavior

- Revert if the caller does not have the burner role.
  - ☑ Negative test
- Revert if the guard contract validation fails.
  - ☑ Negative test
- Revert if the `_from` address has insufficient balance.
  - ☑ Negative test

Function: `mint(address _to, uint256 _amount)`

This function allows a caller with the minter role to mint new tokens and assign them to a specified address.

Inputs

- `_to`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must not be the zero address.
  - **Impact**: The recipient of the minted tokens.
- `_amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.
  - **Impact**: The quantity of tokens to be minted.

Branches and code coverage

Intended branches

- Check if the caller has the minter role.
  - ☑ Test coverage
- Mint the specified amount of tokens to the recipient.
  - ☑ Test coverage
- Return true to indicate success.
  - ☑ Test coverage

Negative behavior

- Revert if the caller does not have the minter role.
  - ☑ Negative test
- Revert if the recipient address is the zero address (standard ERC-20 behavior).
  - ☑ Negative test

Function: `recoverFunds(address _from, address _to, uint256 _amount)`

This function allows a designated `DEFAULT_ADMIN_ROLE` to seize and transfer funds from a nonallowlisted address to a target address.

Inputs

- `_from`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must not be allowlisted by the guard. The address must have a balance of at least `_amount`.
  - **Impact**: The address from which funds are recovered.
- `_to`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must not be the zero address.
  - **Impact**: The recipient of the recovered funds.
- `_amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.
  - **Impact**: The quantity of tokens to be recovered.

Branches and code coverage

Intended branches

- Check if the caller has the `DEFAULT_ADMIN_ROLE`.
  - ☑ Test coverage
- Verify that the `_from` address is not allowlisted via the guard.
  - ☑ Test coverage
- Transfer the specified amount from the source to the recipient.
  - ☑ Test coverage

Negative behavior

- Revert if the caller does not have the `DEFAULT_ADMIN_ROLE`.
  - ☑ Negative test
- Revert if the `_from` address is allowlisted.
  - ☑ Negative test
- Revert if the `_from` address has insufficient balance.
  - ☑ Negative test

Function: `setGuard(address _guard)`

This function allows the `DEFAULT_ADMIN_ROLE` to set the address of the guard contract, which is responsible for validating transfers and checking allowlists.

Inputs

- `_guard`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must not be the zero address.
  - **Impact**: The address of the new guard contract to be used for transfer validation.

Branches and code coverage

Intended branches

- Check if the caller has the `DEFAULT_ADMIN_ROLE`.
  - ☑ Test coverage
- Check if the guard address is valid.

- ☑ Test coverage
- Update the guard contract address in storage.
  - ☑ Test coverage
- Emit the `GuardSet` event.
  - ☑ Test coverage

Negative behavior

- Revert if the caller does not have the `DEFAULT_ADMIN_ROLE`.
  - ☑ Negative test
- Revert if the provided guard address is the zero address.
  - ☑ Negative test

Function: `transfer(address _to, uint256 _amount)`

This function moves tokens from the caller's account to a recipient, subject to additional validation checks performed by the configured guard contract.

Inputs

- `_to`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must not be the zero address.
  - **Impact**: The recipient of the tokens.
- `_amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The caller must have a balance of at least `_amount`.
  - **Impact**: The quantity of tokens to be transferred.

Branches and code coverage

Intended branches

- Verify the transfer with the guard contract.
  - ☑ Test coverage
- Transfer the tokens from the caller to the recipient using the parent implementation.
  - ☑ Test coverage
- Return true to indicate success.

- ☑ Test coverage

Negative behavior

- Revert if the guard contract validation fails.
  - ☑ Negative test
- Revert if the recipient is the zero address.
  - ☑ Negative test
- Revert if the caller has insufficient balance.
  - ☑ Negative test

Function: `transferFrom(address _from, address _to, uint256 _amount)`

This function moves tokens from a sender to a recipient using the allowance mechanism, subject to additional validation checks performed by the configured guard contract.

Inputs

- `_from`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must have a balance of at least `_amount`.
  - **Impact**: The address from which tokens are debited.
- `_to`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The address must not be the zero address.
  - **Impact**: The recipient of the tokens.
- `_amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The caller must have allowance from `_from` of at least `_amount`.
  - **Impact**: The quantity of tokens to be transferred.

Branches and code coverage

Intended branches

- Verify the transfer with the guard contract.
  - ☑ Test coverage
- Check and update the caller's allowance from the sender.

- ☑ Test coverage
- Transfer the tokens from the sender to the recipient using the parent implementation.
  - ☑ Test coverage
- Return true to indicate success.
  - ☑ Test coverage

Negative behavior

- Revert if the guard contract validation fails.
  - ☑ Negative test
- Revert if the recipient is the zero address.
  - ☑ Negative test
- Revert if the sender has insufficient balance.
  - ☑ Negative test
- Revert if the caller has insufficient allowance.
  - ☑ Negative test

5.6. Module: NexusOFT.sol

Function: `send(SendParam _sendParam, MessagingFee _fee, address _refundAddress)`

This function acts as the public entry point for users to send cross-chain token transfers. It handles the payment of LayerZero protocol fees and delegates the core transfer logic to the Nexus contract.

Inputs

- `_sendParam`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: `amountLD > 0`, and `dstEid` must be a valid endpoint ID.
  - **Impact**: Specifies the destination chain, recipient, amount, and execution parameters.
- `_fee`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be sufficient to cover the cross-chain–messaging costs.
  - **Impact**: The fees paid for the transaction.
- `_refundAddress`
  - **Control**: Fully controlled by the caller.

- **Constraints**: N/A.
- **Impact**: The address to receive any excess native gas paid.

Branches and code coverage

Intended branches

- Check if ZRO token fee payment is required.
  - ☑ Test coverage
- Transfer ZRO tokens from the caller to the endpoint if applicable.
  - ☑ Test coverage
- Delegate the cross-chain send operation to the Nexus contract via `nexusSend`.
  - ☑ Test coverage
- Emit the `OFTSent` event with transaction details.
  - ☑ Test coverage

Negative behavior

- Revert if the ZRO token fee is required but the endpoint has no LZ token configured.
  - ☑ Negative test
- Revert if the caller has insufficient ZRO tokens or allowance.
  - ☑ Negative test
- Revert if the underlying `nexusSend` call fails.
  - ☑ Negative test

5.7. Module: NexusOFTAlt.sol

Function: `send(SendParam _sendParam, MessagingFee _fee, address _refundAddress)`

This function acts as the public entry point for users to send cross-chain token transfers on Alt chains.

It handles the payment of LayerZero protocol fees and delegates the core transfer logic to the Nexus contract.

Inputs

- `_sendParam`

    - **Control**: Fully controlled by the caller.
    - **Constraints**: `amountLD > 0`, and `dstEid` must be a valid endpoint ID.
    - **Impact**: Specifies the destination chain, recipient, amount, and execution parameters.
- `_fee`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must be sufficient to cover the cross-chain–messaging costs.
    - **Impact**: The fees paid for the transaction.
- `_refundAddress`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: N/A.
    - **Impact**: The address to receive any excess native gas paid.

Branches and code coverage

Intended branches

- Check if native ERC-20 fee payment is required.
    - ☑ Test coverage
- Transfer native fee from the caller to the endpoint if applicable.
    - ☑ Test coverage
- Check if ZRO token fee payment is required.
    - ☑ Test coverage
- Transfer ZRO tokens from the caller to the endpoint if applicable.
    - ☑ Test coverage
- Delegate the cross-chain send operation to the Nexus contract via `nexusSend`.
    - ☑ Test coverage
- Emit the `OFTSent` event.
    - ☑ Test coverage

Negative behavior

- Revert if `msg.value > 0`.
    - ☑ Negative test
- Revert if the caller has insufficient native ERC-20 tokens or allowance.
    - ☑ Negative test
- Revert if ZRO token fee is required but the endpoint has no LZ token configured.

- ☑ Negative test
- Revert if the underlying `nexusSend` call fails.
  - ☑ Negative test

6. Assessment Results

During our assessment on the scoped LayerZero Console contracts, we discovered 10 findings. No critical issues were found. Three findings were of low impact and the remaining findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.